Write a Custom OAuth 2.0 Grant Type

Introduction

The OAuth 2.0 specification defines a few grant types (authorization code, client credentials, refresh token and so on), and allows extra ones. WSO2 Identity Server lets you add your own grant type with two classes:

Class	What it does
Grant validator	Checks that the token request has all the required parameters. Extends AbstractValidator (Apache Oltu).
Grant handler	Checks the grant (who is the user?) and controls how the token is issued. Extends AbstractAuthorizationGrantHandler.

In this lab you add a **mobile** grant type: the application sends a mobile number to the token endpoint and gets an access token for that user. You work in two parts:

1. **Part 1:** build and test the WSO2 sample **custom-grant** as it is.
2. **Part 2:** improve it so that the mobile number must belong to a real user, and WSO2 Identity Server issues a normal, usable access token.

Note: a grant that trusts a mobile number alone is a teaching example. In a real system the number must be proven, for example with an OTP.

Time: about 60 minutes.

CC BY 4.0

Prerequisites

1. WSO2 Identity Server 7.3.0 (`<IS_HOME>` below) and JDK 21 with `JAVA_HOME` set.
2. Apache Maven 3.6 or later, and Git. The machine needs internet access to `https://maven.wso2.org` and Maven Central, because the build downloads the WSO2 libraries.
3. curl (built into Windows 10 and later, macOS and Linux).
4. `MobileGrant.java` from the **3. Resources** folder of Day 4 (Part 2 only).

Part 1: Build and test the sample

Step 1: Get the sample and read the code

1. Clone the WSO2 samples repository (or download it as a ZIP from GitHub and extract it). The folder is `<SAMPLE_IS>` below:

   ```
   git clone https://github.com/wso2/samples-is.git
   ```

2. The sample is in `<SAMPLE_IS>/oauth2/custom-grant`. The mobile grant classes are in the package `org.wso2.sample.identity.oauth2.grant.mobile`:

   ```
   src/main/java/org/wso2/sample/identity/oauth2/grant/mobile/
   MobileGrant.java
   src/main/java/org/wso2/sample/identity/oauth2/grant/mobile/
   MobileGrantValidator.java
   ```

   The same project also contains two password-grant samples in the `password` package. You do not use them in this lab.

3. The **validator** only lists the required parameter. If a token request does not have `mobileNumber`, WSO2 Identity Server rejects it before the handler runs:

CC BY 4.0

```
public class MobileGrantValidator extends
AbstractValidator<HttpServletRequest> {

    public MobileGrantValidator() {
        // mobile number must be in the request parameter
        requiredParams.add(MobileGrant.MOBILE_GRANT_PARAM);
    }
}
```

4. The **handler** reads `mobileNumber` from the request and checks its format (10 digits, with an optional country code such as `+94`). If the format is right, it sets the mobile number itself as the user of the token. If not, it adds the response header `SampleHeader-999` and rejects the grant:

```
public class MobileGrant extends AbstractAuthorizationGrantHandler {

    public static final String MOBILE_GRANT_PARAM = "mobileNumber";

    @Override
    public boolean validateGrant(OAuthTokenReqMessageContext
oAuthTokenReqMessageContext)
            throws IdentityOAuth2Exception {
        log.info("Mobile Grant handler is hit");
        ...  // read mobileNumber from getRequestParameters()
        authStatus = isValidMobileNumber(mobileNumber);   // regular
expression only
        if (authStatus) {
            AuthenticatedUser mobileUser = AuthenticatedUser
                    .createLocalAuthenticatedUserFromSubjectIdentifier(
tenantAwareUsername);
            oAuthTokenReqMessageContext.setAuthorizedUser(mobileUser);
            ...
        } else {
            // response header SampleHeader-999: Provided Mobile Number
is Invalid.
        }
        return authStatus;
    }
```

CC BY 4.0

5. The handler also overrides `issue()`. Instead of the normal WSO2 Identity Server token, it returns a random value with the token type `mobile`. Keep this in mind for Step 4:

```
    @Override
    public OAuth2AccessTokenRespDTO issue(OAuthTokenReqMessageContext
tokReqMsgCtx) {
        OAuth2AccessTokenRespDTO tokenRespDTO = new
OAuth2AccessTokenRespDTO();

tokenRespDTO.setExpiresIn(tokReqMsgCtx.getAccessTokenIssuedTime() +
10000);
        tokenRespDTO.setAccessToken(UUID.randomUUID().toString());
        tokenRespDTO.setRefreshToken(UUID.randomUUID().toString());
        tokenRespDTO.setTokenType("mobile");
        return tokenRespDTO;
    }
```

Step 2: Build and deploy

1. Build the sample:

```
cd <SAMPLE_IS>/oauth2/custom-grant
mvn clean install
```

2. When the build finishes with `BUILD SUCCESS`, the JAR is in the `target` folder: `custom-grant-<version>.jar` (for example `custom-grant-4.6.5-SNAPSHOT.jar`).
3. Stop WSO2 Identity Server and copy the JAR into `<IS_HOME>/repository/components/lib` (not `dropins`: it is a plain JAR, and WSO2 Identity Server turns it into an OSGi bundle at startup).
4. Open `<IS_HOME>/repository/conf/deployment.toml` and add at the end:

```
[[oauth.custom_grant_type]]
name = "mobile"
grant_handler =
```

CC BY 4.0

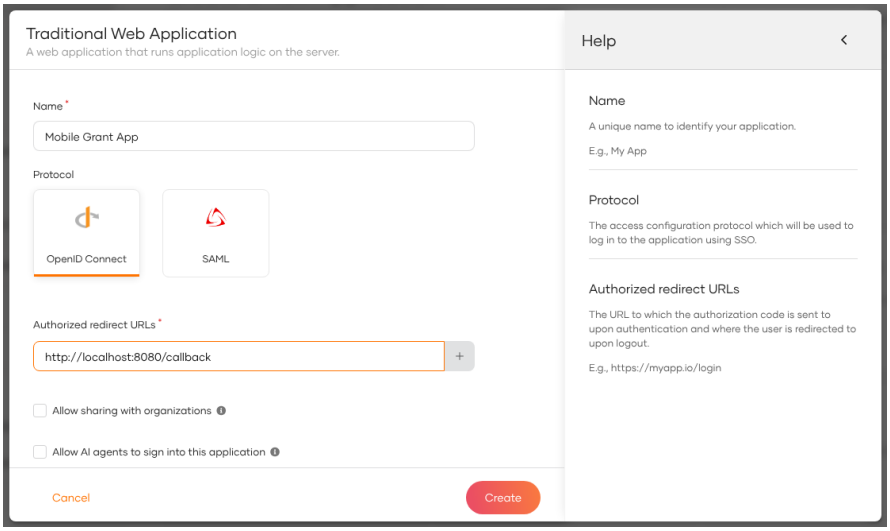
```
"org.wso2.sample.identity.oauth2.grant.mobile.MobileGrant"
grant_validator =
"org.wso2.sample.identity.oauth2.grant.mobile.MobileGrantValidator"
[oauth.custom_grant_type.properties]
IdTokenAllowed = true
PublicClientAllowed = true
```

`name` is the value of `grant_type` in the token request. `grant_handler` and `grant_validator` are the full class names. `IdTokenAllowed` lets the grant return an ID token when the `openid` scope is requested, and `PublicClientAllowed` lets public clients (without a client secret) use it.

5. Start WSO2 Identity Server: `wso2server.bat` (Windows) or `sh wso2server.sh` (Linux and macOS) from `<IS_HOME>/bin`.

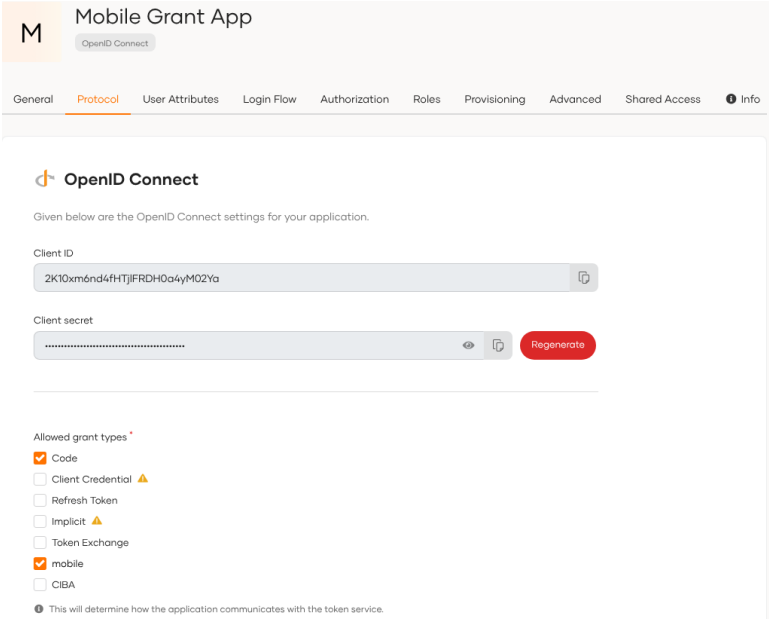
Step 3: Allow the grant type for an application

1. Log in to the Console (`https://localhost:9443/console`) as `admin` / `admin`.
2. Go to **Applications > + New Application > Traditional Web Application**. Enter the name `Mobile Grant App`, select **OpenID Connect**, enter `http://localhost:8080/callback` as the **Authorized redirect URL** (it is not used in this lab) and click **Create**.



CC BY 4.0

3. On the **Protocol** tab, the new grant type **mobile** is listed under **Allowed grant types**. Tick it and click **Update**.



CC BY 4.0

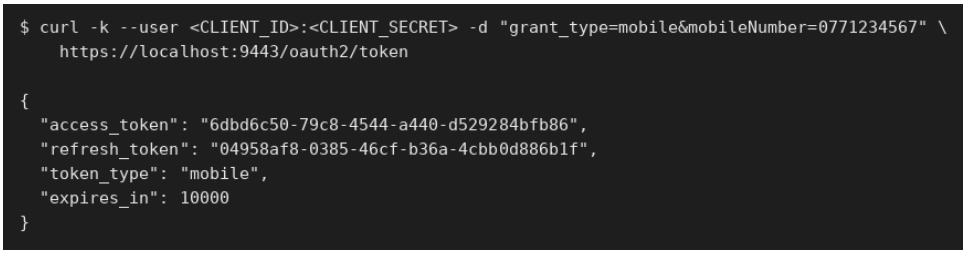
4. Copy the **Client ID** and **Client secret**. Replace `<CLIENT_ID>` and `<CLIENT_SECRET>` with them in the commands below.

Step 4: Try the sample

1. Request a token with a valid mobile number. Each command is one line, so it works the same in Windows Command Prompt and in a Linux or macOS terminal:

```
curl -k --user <CLIENT_ID>:<CLIENT_SECRET> -d
"grant_type=mobile&mobileNumber=0771234567" https://localhost:9443/
oauth2/token
```

   You get a token with the token type **mobile**. The server log shows `Mobile Grant handler is hit`.



CC BY 4.0

2. Send a number with the wrong format. Add `-i` to see the response headers:

```
curl -k -i --user <CLIENT_ID>:<CLIENT_SECRET> -d
"grant_type=mobile&mobileNumber=12345" https://localhost:9443/oauth2/
token
```

The handler rejects it: `400`, the header `SampleHeader-999` and `invalid_grant`.

```
$ curl -k -i --user <CLIENT_ID>:<CLIENT_SECRET> -d "grant_type=mobile&mobileNumber=12345" \
    https://localhost:9443/oauth2/token
HTTP/1.1 400
SampleHeader-999: Provided Mobile Number is Invalid.
...
{"error_description":"Provided Authorization Grant is invalid","error":"invalid_grant"}
```

CC BY 4.0

3. Send the request without `mobileNumber`. This time the **validator** rejects it, before the handler runs:

```
curl -k --user <CLIENT_ID>:<CLIENT_SECRET> -d "grant_type=mobile"
https://localhost:9443/oauth2/token
```

   Response: `{"error_description":"Missing parameters: mobileNumber","error":"invalid_request"}`

4. Now check the token from step 1 with the introspection endpoint (use your own token value):

```
curl -k -u admin:admin -d "token=<ACCESS_TOKEN>" https://
localhost:9443/oauth2/introspect
```

```
$ curl -k -u admin:admin -d "token=6dbd6c50-79c8-4544-a440-d529284bfb86" \
    https://localhost:9443/oauth2/introspect

{"active":false}
```

CC BY 4.0

The answer is `{"active":false}`. Look at the server log from step 1 too:

```
[2026-11-14 17:10:15,205]  INFO {...MobileGrant} - Mobile Grant handler is hit
[2026-11-14 17:10:15,254]  ERROR {...AccessTokenEventUtil} - Error retrieving user Id for
    tenant: carbon.super ... UserIdNotFoundException: User id is not available for user.
```

CC BY 4.0

What this shows: the sample has two limitations. It accepts any number with the right format, even if no user has it, so the "user" of the token is just the number (hence the `User id is not available` error). And its `issue()` method returns a random value that WSO2 Identity Server never stores, so no API can accept it. Part 2 fixes both.

Part 2: Issue real tokens for real users

Step 5: Replace the handler

The improved `MobileGrant.java` (in **3. Resources**) changes `validateGrant()` and removes `issue()`. The main part:

```
        // 2. Find the user whose Mobile attribute has this number.
        int tenantId = IdentityTenantUtil.getTenantId(tenantDomain);
        AbstractUserStoreManager userStoreManager =
(AbstractUserStoreManager) IdentityTenantUtil
                .getRealmService().getTenantUserRealm(tenantId).getUserSto
reManager();
        List<User> users = userStoreManager.getUserListWithID(
                "http://wso2.org/claims/mobile", mobileNumber, null);
        if (users == null || users.size() != 1) {
            return reject(tokReqMsgCtx, "No user found for the mobile
number.");
        }

        // 3. Set that user as the user of the token.
        User user = users.get(0);
        AuthenticatedUser authenticatedUser = new AuthenticatedUser();
        authenticatedUser.setUserId(user.getUserID());
        authenticatedUser.setUserName(user.getUsername());
        authenticatedUser.setUserStoreDomain(user.getUserStoreDomain());
        authenticatedUser.setTenantDomain(tenantDomain);

authenticatedUser.setAuthenticatedSubjectIdentifier(user.getUsername());
```

CC BY 4.0

```
        tokReqMsgCtx.setAuthorizedUser(authenticatedUser);
```

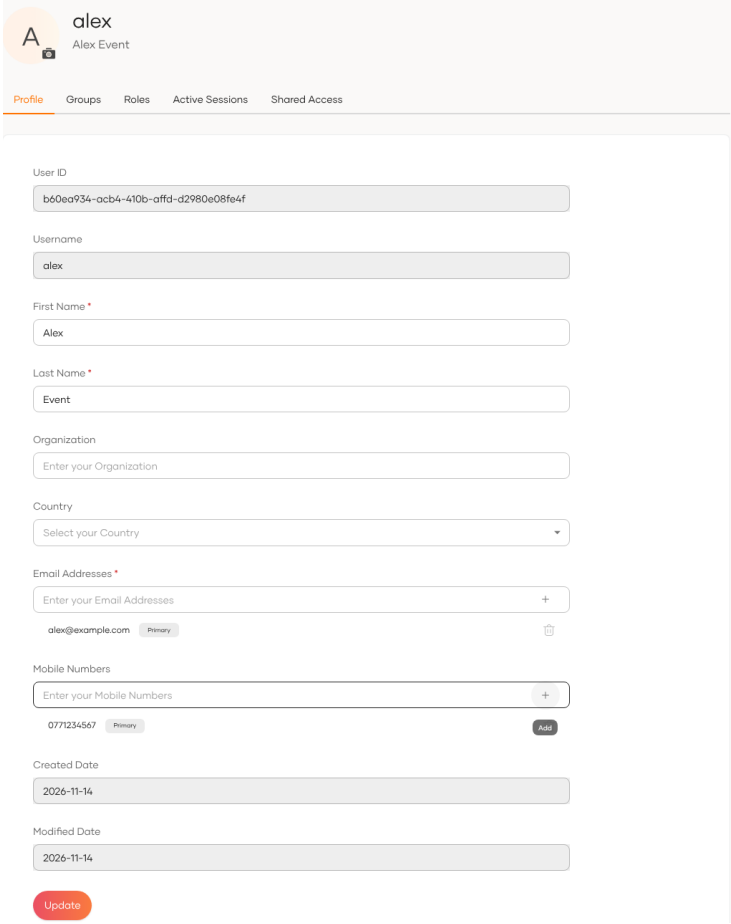
Because `issue()` is no longer overridden, the parent class issues a normal WSO2 Identity Server access token, stored and valid like any other.

1. Copy `MobileGrant.java` from **3. Resources** over `<SAMPLE_IS>/oauth2/custom-grant/src/main/java/org/wso2/sample/identity/oauth2/grant/mobile/MobileGrant.java`.
2. Build again with `mvn clean install`.
3. Stop WSO2 Identity Server, copy the new `custom-grant-<version>.jar` over the old one in `<IS_HOME>/repository/components/lib`, and start the server. `deployment.toml` stays the same.

Step 6: Give a user a mobile number

1. In the Console, go to **User Management > Users > Add User > Single User** and create the user `alex` (email `alex@example.com`, first name `Alex`, last name `Event`, password `Wso2@1234`). If `alex` already exists (from the event handler lab), use it.
2. Open `alex`. In **Mobile Numbers**, enter `0771234567`, click **+** (it is marked **Primary**) and click **Update**.

CC BY 4.0


A
alex
Alex Event
Profile
Groups
Roles
Active Sessions
Shared Access
User ID
b60ea934-acb4-410b-affd-d2980e08fe4f
Username
alex
First Name *
Alex
Last Name *
Event
Organization
Enter your Organization
Country
Select your Country
Email Addresses *
Enter your Email Addresses
alex@example.com
Primary
Mobile Numbers
Enter your Mobile Numbers
0771234567
Primary
Add
Created Date
2026-11-14
Modified Date
2026-11-14
Update

CC BY 4.0

Step 7: Try the improved grant

1. Request a token with Alex's mobile number:

```
curl -k --user <CLIENT_ID>:<CLIENT_SECRET> -d
"grant_type=mobile&mobileNumber=0771234567" https://localhost:9443/
oauth2/token
```

This time the token type is **Bearer** and it expires in 3600 seconds: a normal WSO2 Identity Server token.

```
$ curl -k --user <CLIENT_ID>:<CLIENT_SECRET> -d "grant_type=mobile&mobileNumber=0771234567" \
    https://localhost:9443/oauth2/token

{
  "access_token": "d78c7826-9ff7-3afc-96da-1a304c37a0dc",
  "token_type": "Bearer",
  "expires_in": 3600
}
```

CC BY 4.0

2. Introspect the new token:

```
curl -k -u admin:admin -d "token=<ACCESS_TOKEN>" https://
localhost:9443/oauth2/introspect
```

```
$ curl -k -u admin:admin -d "token=d78c7826-9ff7-3afc-96da-1a304c37a0dc" \
    https://localhost:9443/oauth2/introspect

{
  "aut":"APPLICATION_USER",
  "aud":"2K10xm6nd4fHTjlFRDH0a4yM02Ya",
  "nbf":1794656697,
  "active":true,
  "token_type":"Bearer",
  "exp":1794660297,
  "iat":1794656697,
  "client_id":"2K10xm6nd4fHTjlFRDH0a4yM02Ya",
  "username":"alex@carbon.super"
}
```

CC BY 4.0

The token is **active** and belongs to **alex**.

3. Try a number with the right format that no user has:

```
curl -k -i --user <CLIENT_ID>:<CLIENT_SECRET> -d
"grant_type=mobile&mobileNumber=0779999999" https://localhost:9443/
oauth2/token
```

```
$ curl -k -i --user <CLIENT_ID>:<CLIENT_SECRET> -d "grant_type=mobile&mobileNumber=0779999999" \
    https://localhost:9443/oauth2/token
HTTP/1.1 400
SampleHeader-999: No user found for the mobile number.
...
{"error_description":"Provided Authorization Grant is invalid","error":"invalid_grant"}
```

CC BY 4.0

4. The server log shows both decisions:

```
[2026-11-14 17:14:57,557]  INFO {...MobileGrant} - Mobile Grant handler is hit
[2026-11-14 17:14:57,751]  INFO {...MobileGrant} - Mobile number 0771234567 belongs to user alex
[2026-11-14 17:14:57,778]  INFO {...MobileGrant} - Mobile Grant handler is hit
[2026-11-14 17:14:57,781]  INFO {...MobileGrant} - No single user found with mobile number 0779999999
```

CC BY 4.0

Troubleshooting

Problem	Cause and fix
mvn clean install fails with Could not resolve dependencies / Non-resolvable parent POM	The machine cannot reach https://maven.wso2.org or Maven Central. Check the internet connection or the proxy settings in <MAVEN_HOME>/conf/settings.xml.
mobile is not listed under Allowed grant types	The [[oauth.custom_grant_type]] section is missing or has a typo, the JAR is not in components/lib, or the server was not restarted.
{"error":"unsupported_grant_type"}	The grant type is not ticked for this application (Step 3), or grant_type in the request is not exactly mobile.
{"error":"invalid_client"}	Wrong client ID or client secret in --user.
Missing parameters: mobileNumber	The request has no mobileNumber parameter (the validator rejects it).
Part 2: No user found for the mobile number, although the user has the number	The number was not saved: click + next to the field and then Update. The number must match exactly (0771234567, no spaces), and only one user may have it.
Part 2: still token_type mobile	The old JAR is still in components/lib, or the server was not restarted after copying the new one.

CC BY 4.0